AUDIT TECHNIQUE, FONCTIONNEL & SÉCURITÉ

Bloc 1 — Compétence C2

Epitech Innovative Project
Solution Track - Video Games

Ce document constitue le compte-rendu d'audit réalisé dans le cadre du Bloc 1 de la certification RNCP (compétence C2). Il analyse l'environnement technique, fonctionnel et de sécurité dans lequel s'inscrit le projet, et met en lumière les contraintes et opportunités identifiées.

Table des matières

1. Introduction et périmètre de l'audit....3
2. Approche méthodologique....4
3. Registre de décisions....6
4. Audit de l'environnement technique....10
5. Audit fonctionnel - Benchmark concurrentiel....13
6. Audit sécurité & conformité légale....15
7. Audit accessibilité....17
8. Audit des compétences humaines (RH)....18
9. Synthèse, contraintes et opportunités....20
10. Bibliographie....22

1. Introduction et périmètre de l’audit

Ce document présente l'audit complet réalisé en amont du développement de notre Epitech Innovative Project (EIP), un jeu vidéo RPG action en pixel art développé pour PC. L'équipe est composée de 5 développeurs qui assurent conjointement le design, la programmation, la direction artistique et la gestion de projet.

Le projet met en scène un androïde vivant dans la citadelle-monde d'Albetri, cité technologiquement avancée où les robots sont asservis à une conscience collective digitale appelée la Grande Intelligence. Après un incident, le robot est arraché à sa ville natale et découvre un monde extérieur riche, contredisant la propagande de la cité. Le jeu est fondé sur des choix narratifs induisant plusieurs fins ordonnées, révélant progressivement la vérité sur l’univers.

Périmètre de l’audit

- Axe Technique : stack technologique, moteur, langage, outils de production.
- Axe Fonctionnel : benchmark concurrentiel, analyse des mécaniques prévues, risques de conception.
- Axe Sécurité & Légal : RGPD, licences, propriété intellectuelle, conformité.
- Axe Accessibilité : standards A11y, contraintes spécifiques au projet.
- Axe RH : compétences requises vs. compétences disponibles, plan de remédiation.

2. Approche méthodologique (B1-C02-2)

L'audit a été conduit selon une démarche structurée combinant recherche documentaire, analyse comparative et
introspection d'équipe.

2.1 Recherche documentaire & état de l'art

Une revue approfondie de la documentation officielle de Godot 4 et de GDExtension a été réalisée afin d'évaluer la faisabilité technique de notre choix de langage (C++). Des ressources de la GDC ont été consultées concernant la conception de jeux narratifs à fins multiples. Des post-mortems de jeux similaires (Undertale, NieR:Automata, Disco Elysium) ont également été analysés.

2.2 Benchmark concurrentiel

Cinq jeux représentatifs du genre ont été sélectionnés et analysés selon une grille de critères définis en équipe, couvrant les axes business et technique. Cette analyse est détaillée dans la section 4.

2.3 Ateliers internes d'équipe

Plusieurs sessions de travail collectives ont permis de cartographier les compétences réelles de l'équipe, d'identifier les zones de risque et de confronter les ambitions narratives aux contraintes de faisabilité.

2.4 Prototypage exploratoire

Un prototype minimal du système de dialogues et de choix a été esquissé sous Godot afin de valider la compatibilité entre GDExtension (C++) et les plugins de dialogue (notamment Dialogic).

2.5 Sources et références consultées

Source	Type	Pertinence
Documentation officielle Godot 4 / GDExtension [1]	Documentation technique	Haute
GDC Talks, Narrative Design and Multiple Endings [2]	Conférences	Haute
Post-mortem NieR:Automata, Undertale, Disco Elysium [3]	Analyses sectorielles	Haute

Source	Type	Pertinence
CNIL - Guide RGPD pour les développeurs [4]	Réglementation	Moyenne
OWASP Top 10 2023 [5]	Sécurité	Moyenne
Game Accessibility Guidelines (GAG) [6]	Accessibilité	Haute
INPI - Recherche de marques [7]	Propriété intellectuelle	Moyenne
SteamSpy & itch.io analytics [8]	Marché	Moyenne

3. Registre de décisions

3.1 Vue d'ensemble de l'architecture

Couche	Technologie choisie	Alternative écartée	Criticité
Moteur de jeu	Godot 4	Unity / Unreal Engine	Élevée
Langage principal	C++ via GDExtension	GDScript / C#	Élevée
Système de dialogue	Dialogic + flags C++	Ink / système custom pur	Moyenne
Production artistique	Aseprite + krita	Assets tiers sous licence	Moyenne
Cible plateforme	PC (Win/macOS/Linux)	Console / Mobile	Faible

3.2 Décisions détaillées

Moteur de jeu :	**Godot 4**
Contexte	Le projet requiert un moteur 2D performant, exportable sur PC sans frais, et compatible avec du C++ natif. L'équipe de 5 développeurs n'a pas de budget commercial.
Options évaluées	Unity : Performant et documenté, mais royalties [9] à partir d'un certain seuil de revenus (modèle imprévisible en 2024). Unreal Engine : Trop orienté 3D/AAA, royalties 5% au-delà de 1 M$ de revenus [10], overhead technique excessif pour de la 2D. Godot 4 : Open-source, licence MIT [11], aucune redevance, export natif Win/macOS/Linux, performances 2D excellentes.
Décision	Godot 4 (licence MIT) : zéro royalty, export natif multiplateforme, performances 2D optimales pour un RPG pixel art.

Conséquences	Positif	Risque/Mitigation
	0 € de redevance sur toutes les ventes futures	Courbe d'apprentissage Godot 4 non négligeable
	Export Win/macOS/Linux natif sans surcoût	Migration Godot 3 → 4 non triviale si expérience passée
	Performances 2D excellentes pour notre usage	Mitigation : désigner un référent Godot dès le sprint 1

	Écosystème plugins riche (Dialogic, input manager…)	Mitigation : documenter l'environnement de build en interne

Langage :	**C++ via GDExtension**
Contexte	Le système de flags narratifs et de gestion d'état complexe (fins multiples, variables persistantes sur toute la partie) exige des performances déterministes et un contrôle bas niveau de la mémoire. L'équipe possède 3 ans de formation C++ à Epitech.
Options évaluées	GDScript : Simple, rapide à prototyper, natif Godot. Insuffisant pour les systèmes temps-réel critiques à fort volume de flags. C# : Bon compromis performance/lisibilité, mais couche supplémentaire Mono, moins maîtrisé par l'équipe. C++ via GDExtension : Performances maximales, capital compétences existant, contrôle total de l'architecture des systèmes core.
Décision	C++ (GDExtension) pour les systèmes core (flags narratifs, état de jeu, mécanique de combat). GDScript autorisé pour le scripting secondaire (UI, transitions).

Conséquences	Positif	Risque/Mitigation
	Capitalise sur les compétences C++ de toute l'équipe	Temps de build plus long qu'avec GDScript
	Performances déterministes pour le moteur de flags	Débogage cross-language (C++ ↔ Godot) complexe
	Architecture modulaire et maintenable à long terme	Peu de tutoriels GDExtension vs GDScript
	Valeur pédagogique forte pour la certification RNCP	Mitigation : référent technique désigné + doc interne du binding

Système narratif :	**Dialogic + flags C++ custom**
Contexte	Le jeu repose sur des fins multiples ordonnées nécessitant un tracking précis de l'ensemble des choix du joueur sur toute la durée de la partie.

Système narratif :	**Dialogic + flags C++ custom**
Options évaluées	Ink (Inkle Studios) [12] : Langage de scripting narratif puissant, mais couche supplémentaire à intégrer dans Godot, moins documentée en GDExtension. Système custom pur en C++ : Contrôle total, mais charge de développement maximale, hors scope dans les délais EIP. Dialogic [13] + flags C++ : Dialogic gère la présentation des dialogues/choix (bien documenté, compatible Godot 4) ; C++ gère la logique de fin.
Décision	Dialogic pour l'affichage des dialogues et arbres de choix + couche flags/état custom en C++ pour la logique de résolution des fins.

Conséquences	Positif	Risque/Mitigation
	Dialogic réduit la charge de développement UI narrative	Complexité d'intégration Dialogic ↔ C++ à prototyper tôt
	Séparation claire présentation / logique métier	Risque de dette technique si les flags ne sont pas schemaisés
	Flags C++ = performances et persistance garanties	Mitigation : prototype du système de flags dès le sprint 2
	Evolutif : on peut ajouter des fins sans tout refactoriser	Mitigation : documenter le schéma de flags avant toute implémentation

Production artistique :	**assets 100% originaux (Aseprite + Krita)**
Contexte	Le jeu requiert une identité visuelle cohérente en pixel art. L'équipe souhaite éviter tout risque de copyright et affirmer une identité visuelle différenciante.
Options évaluées	Assets tiers sous licence (itch.io, Kenney…) : Rapide, mais risque de ressemblance avec d'autres jeux, identité visuelle diluée. IA générative (Midjourney…) : Rapide, mais statut légal incertain sur les droits d'exploitation commerciale en 2025. 100% original (Aseprite + Krita) : Cohérence visuelle totale, zéro risque copyright, identité unique sur le marché.
Décision	Production artistique 100% originale avec Aseprite [14] (sprites/animations, ~20 €) et Krita [15] (illustrations de fond, gratuit).

Conséquences	Positif	Risque/Mitigation
	Aucun risque de copyright sur les assets visuels	Charge de production artistique sous-estimable en planning
	Identité visuelle unique et cohérente	Nécessite 1-2 membres dédiés avec compétence pixel art
	Aseprite = standard industrie pixel art indépendant	Mitigation : définir les conventions de production dès sprint 1
	Krita = outil professionnel, gratuit et open-source	Mitigation : modéliser le temps artistique dans chaque sprint

3.2 Résumé des risques et décisions

Décision	Niveau de risque	Statut
Godot 4 (MIT)	Moyen	Acté
C++ via GDExtension	Élevé	Acté
Dialogic + C++ flags	Moyen	En cours
Assets 100% originaux	Moyen	Acté

4. Audit de l’environnement technique

4.1 Moteur de jeu : Godot 4

Godot 4 a été retenu après évaluation comparative de trois moteurs (Unity, Unreal Engine, Godot 4). Pour un projet étudiant sans budget commercial ciblant la 2D, deux critères ont été déterminants : le modèle économique et l'adéquation technique au genre.
Sur le plan économique, Godot est distribué sous licence MIT [11] et ne prélève aucune redevance sur les ventes, contrairement à Unity (royalties à partir d'un certain seuil de revenus [9]) ou Unreal Engine (5% au-delà de 1 M$ de revenus [10]). Sur le plan technique, ses performances 2D sont optimales pour un RPG pixel art, et son export natif vers Windows, macOS et Linux ne nécessite aucune configuration additionnelle.
Unity et Unreal ont été écartés : Unity pour son modèle de licensing imprévisible et son overhead inutile sur un projet 2D, Unreal pour son orientation AAA/3D incompatible avec notre scope et ses royalties dissuasives à long terme.

Critère	Évaluation	Statut
Licence MIT (Gratuit, sans royalties)	0€ à perpétuité, aucune redevance	Opportunité
Export PC (Win/Mac/Linux)	Natif, sans configuration complexe	Opportunité
Support C++ (GDExtension)	Disponible mais complexe à configurer	Contrainte
Écosystème plugins	Riche (Dialogic, etc) mais mature	Opportunité
Courbe apprentissage Godot 4	Moyenne (migration depuis Godot 3 non triviale)	Contrainte
Performances 2D	Excellentes pour un pixel art RPG	Opportunité

4.2 Langage : C++ via GDExtension

Le choix du C++ repose sur deux contraintes conjointes : les exigences techniques du projet et les compétences disponibles dans l'équipe.
Le système de flags narratifs (qui doit tracer l'intégralité des choix du joueur sur toute la durée de la partie pour déterminer la fin obtenue) exige des performances déterministes et un contrôle précis de la gestion mémoire que GDScript ne garantit pas à grande échelle. Par ailleurs, l'ensemble de l'équipe dispose de deux ans de formation C++ à Epitech, ce qui rend ce choix productif sur les systèmes core sans temps d'adaptation majeur.

GDScript a été écarté pour ses limites de performance sur des systèmes d'état complexes. C# a été envisagé comme compromis, mais la couche Mono supplémentaire et sa moindre maîtrise dans l'équipe l'ont éliminé. La décision retenue est d'utiliser C++/GDExtension [16] pour les systèmes critiques (flags, combat, architecture narrative [17]), tout en autorisant GDScript pour le scripting secondaire (UI, transitions).

4.3 Production artistique : Pixel Art 100% original

L'ensemble des assets visuels (sprites, animations, tilesets, UI, effets) sera produit intégralement par l'équipe. Ce choix répond à trois impératifs : cohérence visuelle totale, absence de tout risque copyright, et différenciation sur un marché où les assets génériques sont immédiatement reconnaissables.
Les assets tiers sous licence (itch.io, Kenney) ont été écartés pour leur risque d'identité visuelle diluée. Le recours à l'IA générative a été écarté pour le statut légal incertain sur les droits d'exploitation commerciale en 2025.

- **Outil principal identifié :** Aseprite [14] (standard industrie pixel art, licence ~20$)
- **Outil secondaire envisagé :** Krita [15] (gratuit) pour les illustrations de fond
- **Pipeline à définir :** conventions de nommage, formats d'export, résolutions cibles
- **Risque :** sous-estimation du temps de production artistique – à modéliser dans le planning sprint

4.4 Architecture narrative : système de choix et fins multiples

Le cœur du game design repose sur un système de flags et variables narratives qui trace l'intégralité des choix du joueur pour déterminer la fin obtenue. C'est l'une des architectures les plus complexes à implémenter de façon maintenable.
Trois approches ont été évaluées :

Ink (Inkle Studios) : [13] puissant et éprouvé en gamedev indépendant, mais nécessite une couche d'intégration supplémentaire dans Godot, peu documentée en contexte GDExtension.
Système custom pur en C++ : contrôle total, mais charge de développement maximale, incompatible avec les délais EIP.
Dialogic + flags C++ custom : Dialogic [12] gère la présentation des dialogues et arbres de choix (compatible Godot 4, bien documenté) ; la couche C++ custom gère la logique de résolution des fins. Cette séparation présentation / logique métier est maintenable et évolutive.

La décision retenue est cette approche hybride. Elle permet de commencer avec 2 fins et d'en ajouter progressivement sans refactoriser l'ensemble du système, ce qui correspond directement à la mitigation du risque de scope identifié en section 5.4.

4.5 Configuration matérielle cible

Le jeu étant en 2D pixel art, les exigences matérielles restent volontairement modestes. Ce choix est cohérent avec la décision Godot 4 [18] : maximiser l'audience potentielle sans investissement hardware spécifique.

Composant	Configuration minimale	Configuration recommandée
OS	Windows 10 / Ubuntu 20.04 / macOS 11	Windows 11 / Ubuntu 22.04
CPU	Intel Core i3 (2 cœurs, 2.0 GHz)	Intel Core i5 ou équivalent
RAM	4 Go	8 Go
GPU	Carte graphique intégrée (OpenGL 3.3)	GPU dédié discret
Stockage	500 Mo (estimé)	1 Go SSD recommandé

5. Audit fonctionnel - Benchmark concurrentiel

Cinq jeux représentatifs du marché des RPG indépendants à narration forte ont été analysés. Ce benchmark permet de positionner notre projet, d'identifier des opportunités différenciantes et de s'appuyer sur les meilleures pratiques du secteur.

5.1 Tableau comparatif

Jeu	Choix/Fins multipes	Pixel Art	Thème robot/ conscience	Prix	Réception
Undertale [19]	Oui (3 fins)	Oui	Non	9,99 €	Très positive (97% Steam)
NieR:Automata [20]	Oui (26 fins)	Non	Oui (central)	39,99 €	Overwhelming (95%)
Disco Elysium [21]	Oui (5 fins)	Non	Non	39,99 €	Overwhelming (97%)
Omori [22]	Oui (5 fins)	Oui	Non	19,99 €	Overwhelming (96%)
Citizen Sleeper [23]	Oui (3 fins)	Non	Oui (robot libre)	19,99 €	Très positive (93%)
Notre projet	**Oui (TBD)**	**Oui**	**Oui (central)**	**TBD**	**-**

5.2 Analyse des jeux de référence

NieR:Automata (Platinum Games / Square Enix, 2017)

• Référence absolue pour la thématique robot/conscience artificielle et les fins multiples ordonnées.
• Les fins s'enchaînent obligatoirement dans un ordre narratif précis, chaque run révélant une nouvelle couche de vérité : concept très proche de notre vision.
• Leçon clé : la rejouabilité forcée est acceptée par les joueurs si le contenu de chaque run apporte une valeur narrative réelle et distincte.
• Point de vigilance : NieR est un jeu AAA avec un budget de production massif. Notre version pixel art devra justifier sa propre identité visuelle pour éviter une comparaison défavorable.

Undertale (Toby Fox, 2015)

• Démontre qu'un jeu pixel art solo peut atteindre un succès critique et commercial mondial.
• Le système de choix moraux impacte directement la jouabilité (pas uniquement la narration).
• L'onboarding est immédiat et la courbe de difficulté est bien maîtrisée.

Disco Elysium (ZA/UM, 2019)

• Montre jusqu'où peut aller la profondeur narrative dans un RPG indépendant.
• Les choix du joueur sont vraiment conséquents et mémorisés sur toute la durée du jeu.
• Architecture technique complexe à base de flags et de scores de compétences – référence pour notre système de fins.
• L'équipe ZA/UM a rencontré des difficultés techniques importantes en cours de production : un audit de faisabilité rigoureux en amont est essentiel.

5.2 Analyse des jeux de référence

Ce que notre projet apporte d'unique

- Combinaison inédite : thème robot/conscience (NieR) + pixel art total (Undertale) + fins ordonnées révélatrices.
- Mécanisme de manipulation du joueur vers la 'fin basique' : concept peu exploré dans le jeu indépendant.
- Assemblage modulaire du robot (bras, jambes, accessoires) : personnalisation du build sans skill tree classique.
- Narration environnementale forte : la ville 'utopique' vs. l'extérieur 'désolé' renverse les attentes du joueur.

5.3 Risques fonctionnels identifiés

Risque	Probabilité	Impact	Mitigation
Début trop répétitif	Élevée	Élevé	Tester l'onboarding dès le prototype
Système de fins trop complexe à implémenter dans les délais	Moyenne	Élevé	Commencer avec 3 fins, en ajouter progressivement
Comparaison immédiate à NieR:Automata	Moyenne	Moyen	Affirmer l'identité pixel art et la différence du narratif

6. Audit sécurité et conformité légale

6.1 RGPD - Données personnelles

Le jeu est conçu comme une expérience PC solo, sans création de compte obligatoire ni connexion à des serveurs distants dans sa version initiale. Le risque RGPD [24][25] est donc globalement faible, mais plusieurs points méritent une attention particulière.

• Sauvegardes locales : aucune donnée personnelle collectée → conformité naturelle
• Analytics comportementaux (étude des choix automatiques pour guider vers la 'fin basique') : si ces données sont collectées et envoyées à un serveur, elles deviennent des données comportementales soumises au RGPD. Un consentement explicite et une politique de confidentialité seront obligatoires.
• Fonctionnalités futures (leaderboard, cloud saves, multijoueur) : la moindre collecte de données nécessite une analyse d'impact RGPD préalable.
• Recommandation : documenter dès maintenant quelles données sont collectées, pourquoi, et comment elles sont stockées/supprimées.

6.2 Propriété intellectuelle et licences

Élément	Situation	Risque	Action requise
Assets visuels	100% originaux (équipe)	Aucun	Maintenir cette politique
Musiques et SFX	À composer en interne ou licences libres	Moyen si externes	Vérifier toutes les licences [26]
Nom du jeu et studio	Non vérifié à ce stade	Moyen	Recherche INPI [27] + Google Trademark
Code source	Licences MIT / Apache 2.0 [28]	Faible	Mentionner les attributions requises
Inspirations artistiques	Inspiration/références à d'autres oeuvres	Moyen	Éviter les ressemblances trop directes

6.3 Risques de sécurité

Vecteur	Risque	Mitigation
Manipulation des fichiers de sauvegarde [29]	Faible	Vérification d'intégrité (checksum) des saves si succès Steam

Vecteur	Risque	Mitigation
Exécution de code externe (plugins futurs) [29]	Faible à Moyen	Restreindre les sources de plugins, sandbox stricts
Données analytiques (si collecte) [30]	Moyen	Transmission via HTTPS, stockage sécurisé
Télémétrie système non consentie [30]	Moyen	Documenter les collectes, obtenir consentement explicite

7. Audit accessibilité

Afin de rendre le jeu accessible au plus de monde possible nous voulons suivre les lignes directrices proposées par le site Game Accessibility Guidelines [31]. Créé par un ensemble d'académiciens, de spécialistes et de studios de jeux-vidéos, il réunit un ensemble d'éléments pouvant être mis en place afin de rendre un jeu accessible aux personnes en situation de handicap.

7.1 Niveau de conformation

Il existe 3 lignes directrices: Basic, Intermediate, Advanced. En fonction de celle que nous choisissons, cela peut entraîner un temps de travail plus important ou des coûts supplémentaires.

- **Basic :** Cette ligne directrice comprend des considérations simples, rapides à mettre en place et qui s'adaptent à la plupart des mécaniques de jeu. Respecter les conseils qui y sont listés suffit déjà à toucher un public significativement plus large.
- **Intermediate :** Ici on retrouve beaucoup plus de dispositifs à mettre en place. Cela demande plus de planification.
- **Advanced :** Répertorie des adaptations plus complexes et parfois coûteuses. S'y conformer serait vraiment afin de toucher un public niche. De plus les recommandations ne s'appliquent qu'à des mécaniques très spécifiques qui ne sont pas forcément adaptées à notre jeu.

Nous avons donc choisi de suivre la ligne directrice Basic afin de toucher un public assez large tout en minimisant le temps de travail à allouer à la mise en place des spécifications.

7.2 Recommandations à appliquer

Visuelles :
- Contraste élevé entre l'UI et le reste du jeu
- Police d'écriture simple à lire
- Utilisation de palettes de couleurs différentes pour les personnes daltoniennes

Auditives :
- Utilisation de sous-titre clairs et lisibles
- Éviter de transmettre des informations essentielles par le son
- Permettre de paramétrer le volume des effets sonores, des dialogues et de la musique séparément

Motrices :
- Permettre d'ajuster la vitesse du jeu
- Permettre de changer la sensibilité
- Pouvoir changer les contrôles sur manette comme sur clavier

8. Audit des compétences humaines (RH)

Un projet ambitieux comme celui-ci exige une équipe aux compétences complémentaires et bien réparties. Cette section cartographie les compétences nécessaires, les compare aux compétences réelles de l'équipe, et propose un plan d'action pour combler les écarts.

8.1 Matrice des compétences requises vs. disponibles

Compétences	Niveau requis	Niveau équipe (estimé)	Écart	Plan d'action
C++ avancé	Élevé	Moyen	Moyen	Pair programming
Godot 4	Élevé	Faible à Moyen	Élevé	Documentation interne
Pixel Art et Animation	Élevé	Moyen à Élevé	Faible	2-3 membres dédiés, workflows définis
Game Design narratif	Élevé	Moyen	Moyen	Analyse approfondie des oeuvres références
Architecture logicielle	Élevé	Moyen	Moyen	Code reviews systématiques, patterns définis
Sound design/ Composition	Moyen	Faible	Élevé	Formation DAW ou recours à des banques libres
Gestion de projet	Moyen	Moyen à Élevé	Faible	Utilisation de Notion, réunions hebdo, communication Discord
Legal/RGPD	Faible	Faible	Faible	Consultation des ressources CNIL disponibles

8.2 Répartition des rôles

Rôle	Responsabilités principales	Profil
Lead Développeur / Architecte	Architecture C++/Godot, GDExtension, systèmes core	1 membre
Développeur Gameplay	Mécaniques de jeu, combat, système de modules/craft	1 membre
Développeur Narration	Système de choix, flags, fins multiples, Dialogic	2 membres

Rôle	Responsabilités principales	Profil
Lead Artiste / Pixel Art	Sprites, animations, tilesets, cohérence visuelle	2-3 membres
Game Designer	GDD, balancing, planning sprints, coordination	1 membre

9. Synthèse, contraintes et opportunités

Cette section consolide l'ensemble des conclusions de l'audit en une vue synthétique des contraintes à gérer et des opportunités à exploiter pour maximiser les chances de succès du projet.

8.1 Contraintes majeures

Complexité technique C++/GDExtension [Technique]

Le choix du C++ avec GDExtension est le risque technique le plus élevé. Il nécessite une montée en compétences rapide et une organisation technique rigoureuse.

Architecture narrative à fins multiples [Technique / Design]

Un système de flags robuste et évolutif doit être conçu dès le début. Une conception bâclée rendrait le projet inmaintenable.

Charge de production artistique [Production]

Tout réaliser en interne est une contrainte temporelle forte. Le pipeline doit être optimisé et les conventions de production définies tôt.

Accessibilité

Adoption des recommandations Basic du site Game Accessibility Guidelines afin d’améliorer l’accessibilité du jeu (visuelle, auditive et motrice) tout en limitant le coût et le temps de développement.

Son / Sound design sous-représenté [RH]

Aucun membre de l'équipe n'a de compétence forte en composition ou sound design. Ce poste est un gap critique à combler.

8.2 Opportunités stratégiques

Positionnement narratif unique [Marché]

La combinaison fins ordonnées + manipulation psychologique + pixel art + thème robot/conscience est une proposition de valeur sans équivalent direct sur le marché indépendant actuel.

Godot 4 – Zéro frais de licence [Économique]

Aucune redevance sur les ventes futures. L'ensemble du chiffre d'affaires est conservé par l'équipe.

Jeu 2D pixel art – Accessibilité hardware maximale [Marché]

Le jeu fonctionnera sur 95%+ des machines disponibles, maximisant l'audience potentielle sans investissement hardware spécifique.

Écosystème Godot riche en plugins [Technique]

Dialogic, systèmes de sauvegarde, gestionnaires d'input – de nombreuses briques techniques réduisent la charge de développement custom.

Assets 100% originaux [Légal / Artistique]

Aucun risque de copyright, identité visuelle unique, pas de dépendance à des éditeurs d'assets tiers.

10. Bibliographie

[1] Documentation officielle Godot 4 / GDExtension : https://docs.godotengine.org/en/stable/tutorials/scripting/gdextension/
[2] GDC Talks - Narrative Design and Multiple Endings : https://gdcvault.com/browse
[3] Post-mortem NieR:Automata, Undertale, Disco Elysium
[4] CNIL – Guide RGPD pour les développeurs : https://www.cnil.fr/fr/guide-rgpd-du-developpeur
[5] OWASP Top 10 2023 : https://owasp.org/www-project-top-ten/
[6] Game Accessibility Guidelines (GAG) : https://gameaccessibilityguidelines.com/
[7] INPI – Recherche de marques : https://data.inpi.fr/
[8] SteamSpy & itch.io analytics : https://steamspy.com / https://itch.io/docs/creators/analytics
[9] Unity royalties : https://unity.com/pricing
[10] Unreal Engine EULA : https://www.unrealengine.com/en-US/eula/unreal
[11] Licence MIT Godot : https://godotengine.org/license/
[12] Dialogic plugin Godot 4 : https://github.com/dialogic-godot/dialogic
[13] Ink (Inkle Studios) : https://www.inklestudios.com/ink/
[14] Aseprite : https://www.aseprite.org/
[15] Krita : https://krita.org/
[16] Documentation GDExtension : https://docs.godotengine.org/en/stable/tutorials/scripting/gdextension/
[17] Godot-cpp bindings GitHub : https://github.com/godotengine/godot-cpp
[18] Godot system requirements : https://docs.godotengine.org/en/stable/about/system_requirements.html
[19] Undertale : https://store.steampowered.com/app/391540/Undertale/
[20] NieR:Automata : https://store.steampowered.com/app/524220/NieRAutomata_YoRHa_Edition/
[21] Disco Elysium : https://store.steampowered.com/app/632470/Disco_Elysium__The_Final_Cut/
[22] Omori : https://store.steampowered.com/app/1150690/OMORI/
[23] Citizen Sleeper : https://store.steampowered.com/app/1578650/Citizen_Sleeper/
[24] CNIL Guide développeurs : https://www.cnil.fr/fr/guide-rgpd-du-developpeur
[25] Règlement (UE) 2016/679 texte intégral : https://eur-lex.europa.eu/eli/reg/2016/679/oj
[26] Creative Commons CC0 : https://creativecommons.org/publicdomain/zero/1.0/
[27] INPI recherche de marques : https://data.inpi.fr/
[28] Apache License 2.0 : https://www.apache.org/licenses/LICENSE-2.0
[29] OWASP Game Security Framework : https://owasp.org/www-project-game-security-framework/
[30] OWASP Top 10 : https://owasp.org/www-project-top-ten/
[31] Game Accessibility Guidelines : https://gameaccessibilityguidelines.com/